Date

Expt. No. 11. Page No. 34.

Aim:- YACC program to Evaluate Arithmetic expressions using arithmetic operators such as addition (+) subtraction (-), multiplication (*) & division (/)

Algorithm

1. Start the program
2. Define arithmetic grammar rules in YACC.
3. ~~Specify~~ Define tokens for numbers & arithmetic operators using LEX.
4. Specify operator precedence & associativity to evaluate expressions correctly.
5. Evaluate the expression according to the grammar rules & display the result
6. Stop

Code

File 1 exp11.l

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+      {yylval = atoi(yytext);
             return NUMBER;}
[\t]+       ;
\n          {return "\n";}
.           {return yytext[0];}
%%
int yywrap() {
    return 1;
}
```

Teacher's Signature

File 2 : expll.y

```
%{
  #include <stdio.h>
  # include <stdlib.h>
int yylex (void);
void yyerror (const char *s);
 %};

% token NUMBER
%. left '+' '-'
%. left '*' '/'
%. right UMINUS
%.%
  input :
       expr '\n' { printf ("Result = %d\n", $1); }
       ;

  expr :
      NUMBER          { $$ = $1; }
    | expr '+' expr   { $$ = $1 + $3; }
    | expr '-' expr   { $$ = $1 - $3; }
    | expr '*' expr   { $$ = $1 * $3; }
    | expr '/' expr   { $$ = $1 /$.
                  { if ( $3 == 0){
                      yyerror ("Division by zero");
                      $$ = 0;
                  } else {
                      $$ = $1 / $3; }
                  }
```

Teacher's Signature _____

```
| '-' expr %prec UMINUS { $$ = -$2; }
| '(' expr ')' { $$ = $2; }
;
%%
void yyerror(const char *s)
{
printf("Error : %s\n", s);
}
int main()
{ printf("Enter an arithmetic expression:");
yyparse();
return 0;
}
```

Viva Question

Q1 What is YACC?
A1 YACC stands for Yet Another Compiler Compiler. It generates a parser from grammar rules to analyse the syntax of input.

Q2 What is ~~the~~ purpose of yyparse()?
A2 yyparse() starts the parsing process to check the input against the grammar rules.

Q3 What is the purpose of yyerror()?
A3 yyerror() is used to display error messages when a syntax or parsing error occurs.

Date

Expt. No. _____ Page No. 32

Q4 What is the role of LEX in the program?

Ans LEX identifies numbers, whitespaces, newlines & operators, then passes the recognized tokens to YACC.

Q5 What is the difference between YACC & LEX?

Ans LEX performs Lexical Analysis & generates tokens, whereas YACC performs syntax analysis & applies grammar rules to evaluate expressions

Teacher's Signature _____

Date

Expt. No. 12. Page No. 38.

Aim To write & implement a YACC program to check whether a given input string belongs to a specified grammar or not.

Algorithm

1. Start
2. Define the grammar rules in YACC program.
3. Specify the valid token & pattern using LEX.
4. Read the input from user.
5. Pass the token generated by LEX to YACC parser.
6. Check whether the input string follows the grammar rules.
7. If string is valid, display "valid". Otherwise "invalid".

Code for example.

$S \rightarrow aSb \mid ab$

accepted ⇒ ab, aabb, aaabbb.

File 1 (LEX)

```
%{ #include "y.tab.h"
%}
%%
[a]   { return A; }
[b]   { return B; }
[\n]  { return '\n'; }
[\t]  ;
.     { return INVALID; }
%%
```

Teacher's Signature

```
int yywrap()
{
return 1;
}
```

File 2 (YACC)

```
%{ #include <stdio.h>
int yylex (void)
void yyerror (const char *s);
%}
%token A B INVALID
%%
input :
    S '\n' { printf ("String is valid.\n"); }
    ;
S:
    A S B
    | A B ;
%%
void yyerror(const char *s)
{
    printf ("String is invalid.\n");
}
int main()
{
    printf (" Enter a string:");
    yyparse();
    return 0;
}
```

Teacher's Signature _____

Date

Expt. No. Page No. 40.

Viva Question

Q1 What is a grammar?

Ans A grammar is a set of production rules used to define the valid structure of strings in a language.

Q2 What is the purpose of this experiment?

Ans The purpose is to check whether an input string follows the specific grammar.

Q3 What is a production rule?

Ans A production rule defines how a non-terminal symbol can be replaced by terminals or other non-terminals.

Q4 What is a terminal symbol?

Ans A terminal is a basic symbol of a grammar that cannot be replaced / expended further, such as a, b.

Q5 What is a non-terminal symbol?

Ans A non-terminal symbol is a grammar symbol that can be expended using production rules such as S.

Teacher's Signature

Date

Expt. No. 13. Page No. 41.

Aim To write a program to calculate the first set of a given context-free grammar.

Algorithm

1. Start
2. Read the grammar productions from the user
3. Identify the terminal & non-terminals in grammar.
4. If a production starts with terminal, add that terminal to first set.
5. If a production starts with non-terminal,
6. find its first set recursively.
6. If a production can derive epsilon (ε), include epsilon when appropriate.
7. Display the first set of each non-terminal.
8. End.

Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char productions[10][10];
char first[10];
int n, count;
void add(char c)
  {
    for(int i=0; i<count; i++)
      { if(first[i]==c)
        return; }
    first[count++]=c;
  }
```

Teacher's Signature

```
void findFirst(char c)
{
  if(!isupper(c)){
      add(c);
      return;
    }
for(int i=0; i<n; i++){
    if(productions[i][0] == c){
        char *rhs = productions[i]+3;
      if(*rhs == '#'){
          add('#');
          continue;
        }
for(int j=0; rhs[j] != '\0'; j++){
    char oldFirst[10];
    int oldCount = count;

    findFirst(rhs[j]);

    if(first[count-1] == '#')
      count = oldCount;
    if(j==0 && !isupper(rhs[j]))
      break;
    if(j==0)
      break;
    }
  }
 }
}
```

```
int main()
{ char c;
  printf("Enter number of productions: ");
  scanf("%d", &n);
  printf("Enter productions");;
  for(int j=0; j<n; j++):
      scanf("%s', productions[i]);

printf("\nFirst sets:\n");

for(int i=0; i<n; i++){
    c = production[i][0];
    count = 0;
    findfirst(c);
  printf("First(%c) = {", c);

for(int j = 0; j< count; j++)
      printf("%c", first[j]);
  printf("}\n");
 }
return 0;
}
```

Date

Expt. No. _____ Page No. 1

Viva Question

Q1 What is FIRST in Compiler Design?

Ans FIRST is the set of terminals that can appear at the beginning of the strings derived from a grammar symbol.

Q2 What is epsilon (ε)?

Ans Epsilon represents the empty string, meaning a production can generate no symbol.

Q3 Why is the FIRST Set important?

Ans The FIRST set helps construct predictive parsing tables to determine which production rule a parser should select.

Q4 When is epsilon included in the FIRST set?

Ans Epsilon(ε) is included when a non-terminal can derive the empty string without producing any terminal symbols.

Q5 What is left Recursion?

Left recursion occurs when a non-terminal derives a string beginning with itself, such as $A \rightarrow A\alpha$. It can cause problems for predictive parsers.

Teacher's Signature _____